

DEEP LEARNING

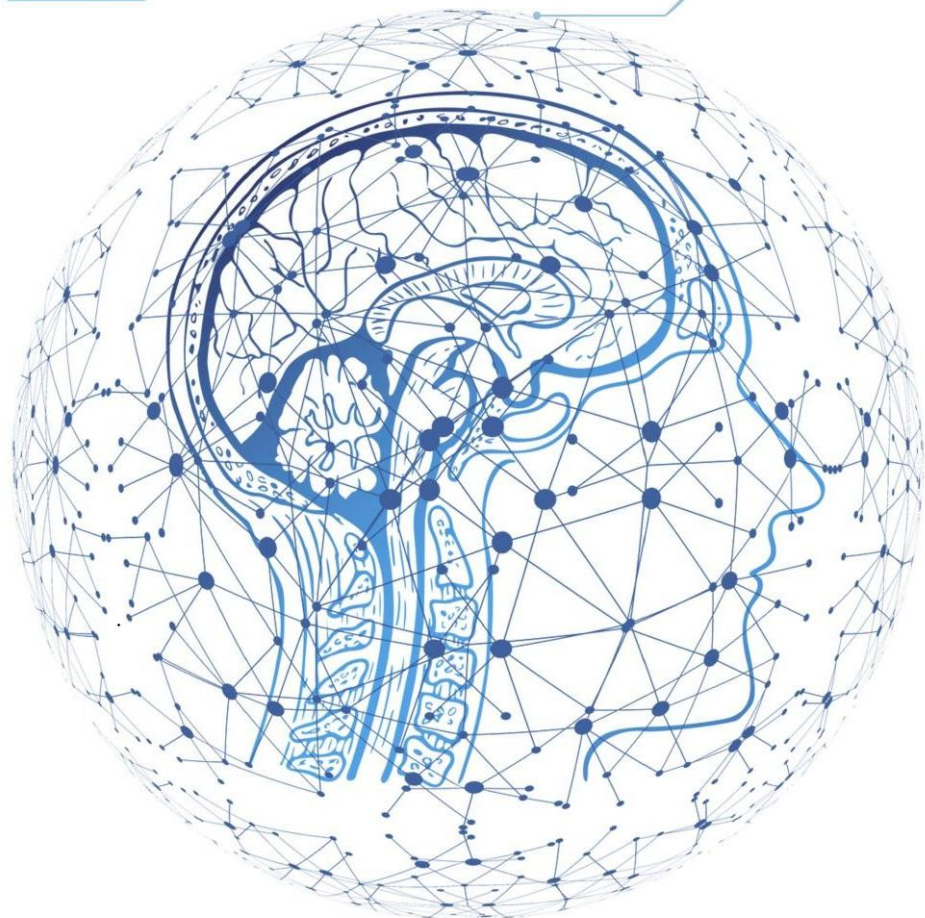
NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

www.hoasen.edu.vn

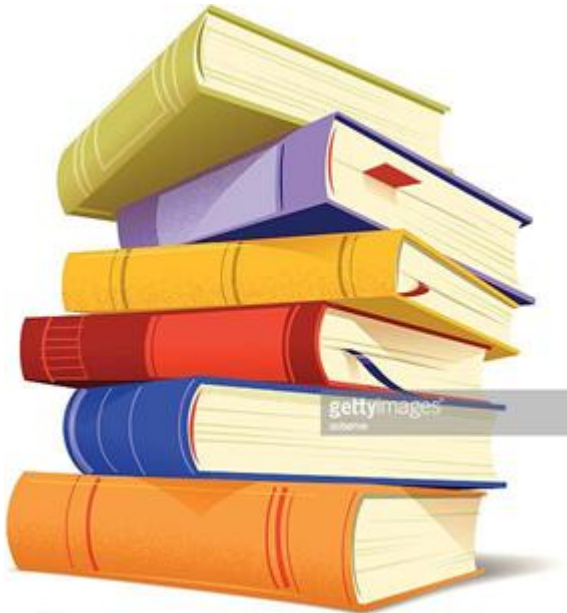
Chương 5

Những nguyên lý cơ bản của học máy

Fundamentals of Machine Learning



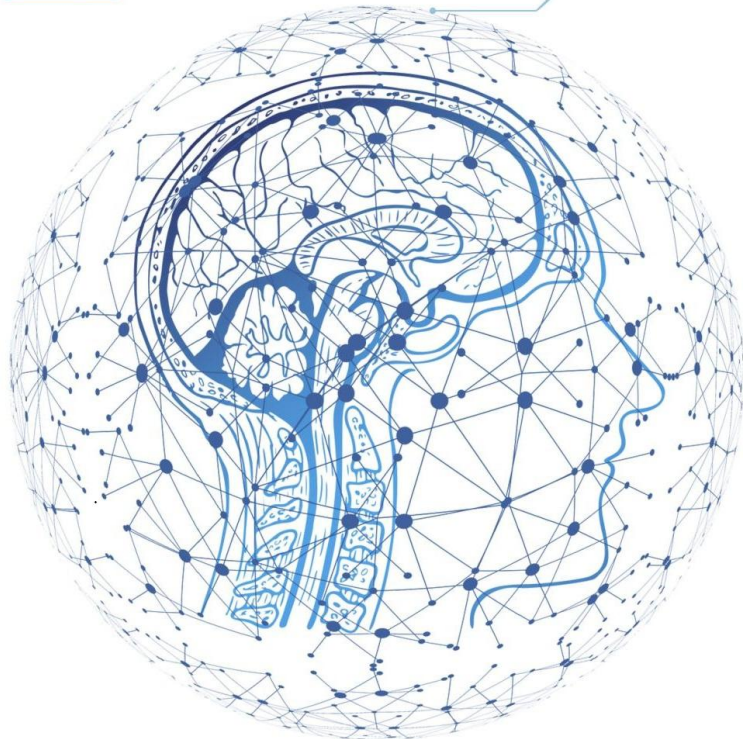
Những nguyên lý cơ bản của học máy



- Khái quát hóa: Mục tiêu của học máy
- Đánh giá các mô hình học máy
- Cải thiện độ phù hợp của mô hình
- Cải thiện khả năng khái quát hóa

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Khái quát hóa:
Mục tiêu của học máy
Generalization: The goal of machine learning

Khái quát hóa: Mục tiêu của học máy

- Mục tiêu cuối cùng của machine learning là **generalization**
- Generalization:
 - Mô hình hoạt động tốt trên **dữ liệu chưa từng thấy**
 - Không chỉ tối ưu trên dữ liệu train

Mâu thuẫn giữa tối ưu hóa và khái quát hóa

- **Optimization:** làm mô hình khớp tốt với training data
- **Generalization:** mô hình hoạt động tốt trên new data
- Hai mục tiêu này **liên quan nhưng không hoàn toàn giống nhau**

Underfitting, Robust Fit, Overfitting

■ Underfitting - Dưới khớp:

- mô hình còn quá đơn giản
- chưa học được quy luật chính

■ Robust fit – Khớp vững chắc/bền vững:

- mô hình học được quy luật quan trọng

■ Overfitting – Quá khớp:

- mô hình học cả nhiễu và đặc thù riêng của tập train

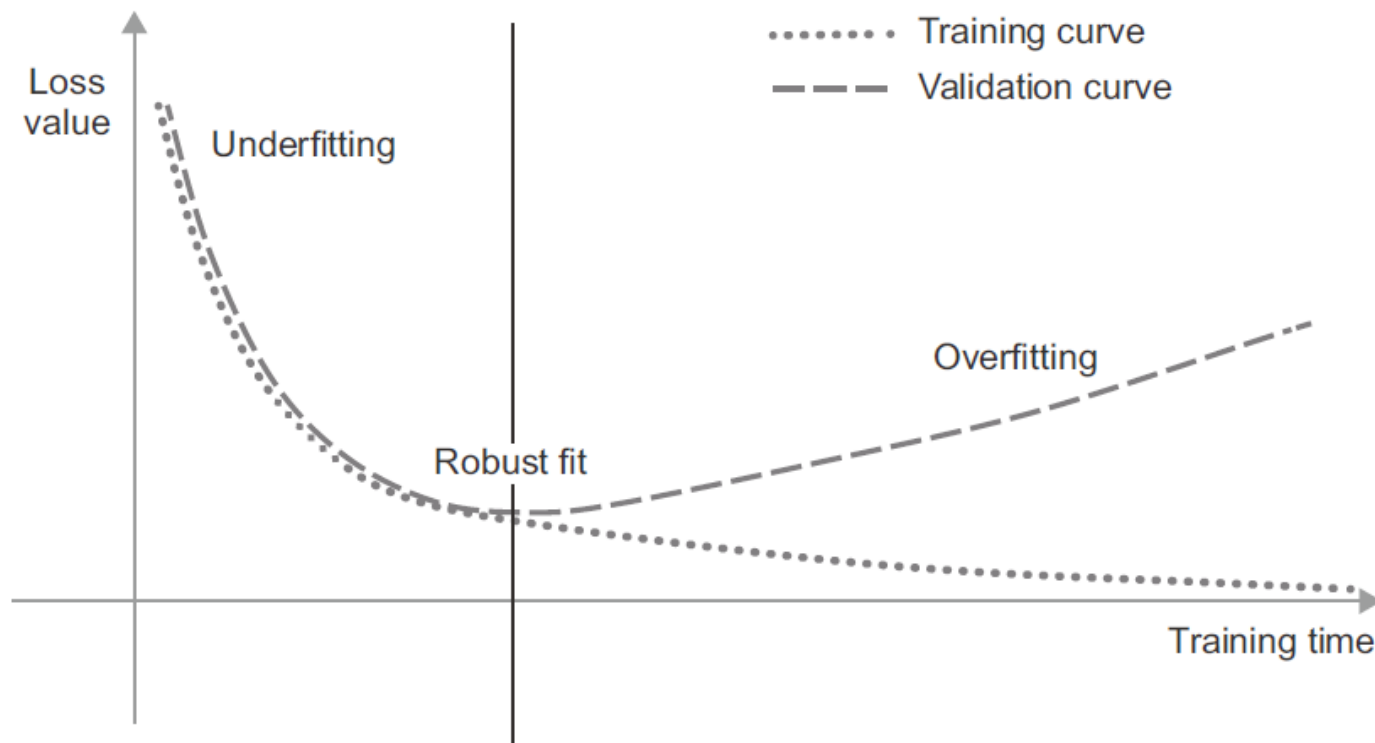


Figure 5.1 Canonical overfitting behavior

Dấu hiệu của Underfitting

- Training loss còn cao
- Validation loss cũng cao
- Cả train và validation đều chưa tốt
- Mô hình chưa đủ khả năng học dữ liệu

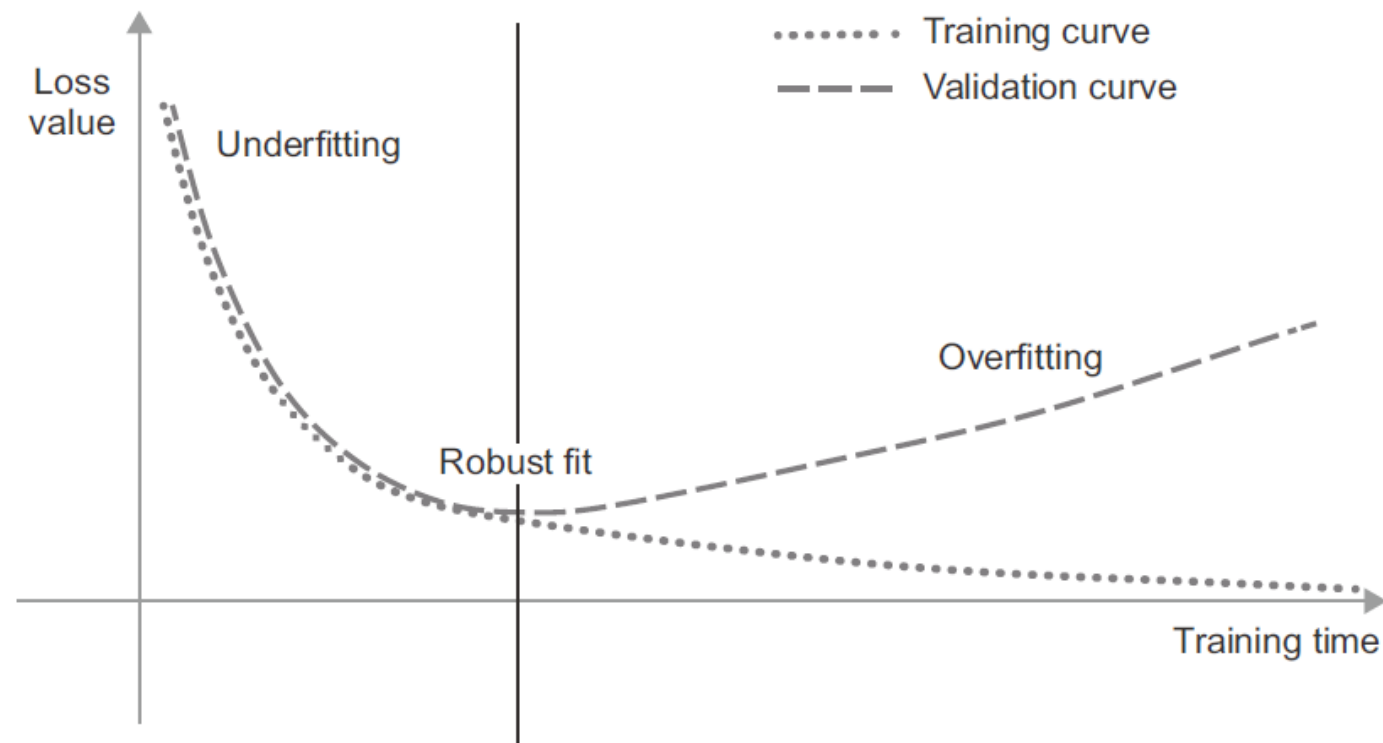


Figure 5.1 Canonical overfitting behavior

Dấu hiệu của Overfitting

- Training loss tiếp tục giảm
- Validation loss ngừng cải thiện rồi tăng
- Train accuracy rất cao nhưng validation/test không theo kịp

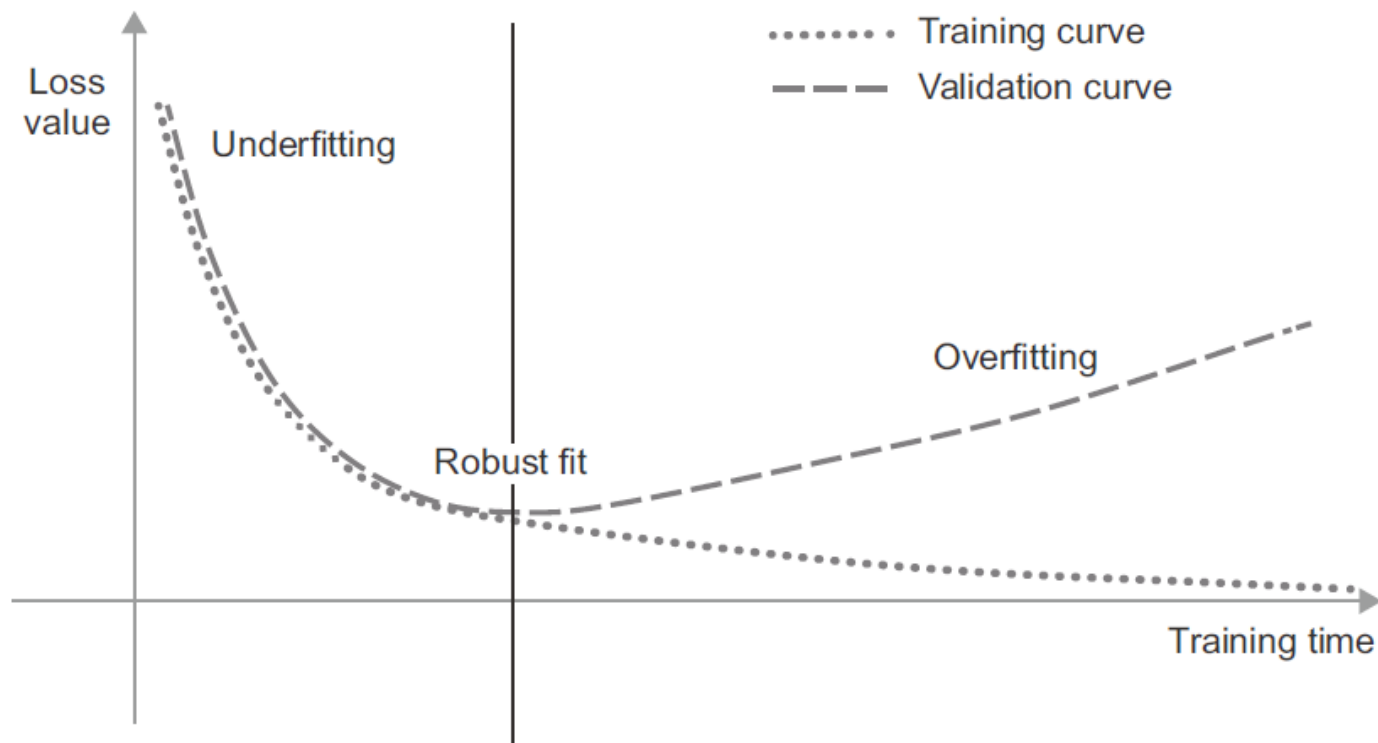


Figure 5.1 Canonical overfitting behavior

Vì sao Overfitting xảy ra?

- Dữ liệu thực tế thường có:

1. noise – nhiễu: dữ liệu lỗi, bất thường, ngoại lệ.

- Ví dụ: ảnh nhìn giống số 4 nhưng label lại ghi là 9.

2. uncertainty - không chắc chắn: dữ liệu mơ hồ, nhiễu hoặc nhập nhằng; ngay cả con người cũng có thể không chắc label đúng là gì.

- Ví dụ: ảnh bị mờ.

3. rare features – đặc trưng hiếm: các đặc trưng xuất hiện quá ít trong training set, khiến model dễ học nhầm các tín hiệu không ổn định hoặc không đại diện.

- Ví dụ: một cụm từ đặc biệt chỉ có trong vài review phim.

4. spurious correlations – mối tương quan giả

tạo: một số quan hệ chỉ là tình cờ trong training data, nhưng model có thể hiểu nhầm là qui luật thật.

- Ví dụ: trong training data, nhiều ảnh chó được chụp ngoài trời, nhiều ảnh mèo được chụp trong nhà. Model có thể học nhầm mối qui luật: ngoài trời → chó; trong nhà → mèo

- Mô hình có thể học nhầm các mẫu này như thể đó là quy luật thật → Dễ tạo ra kết luận giả cho mô hình

Liên hệ với overfitting

- Nếu model quá phức tạp, nó có thể cố gắng học cả những mẫu kỳ lạ này.
- Vấn đề là:
 - Những mẫu này không đại diện cho quy luật chung.
 - Chúng có thể là outliers hoặc noise.
 - Nếu model học quá kỹ chúng, model sẽ dễ overfit.
 - Khi gặp dữ liệu mới, model có thể dự đoán kém hơn.

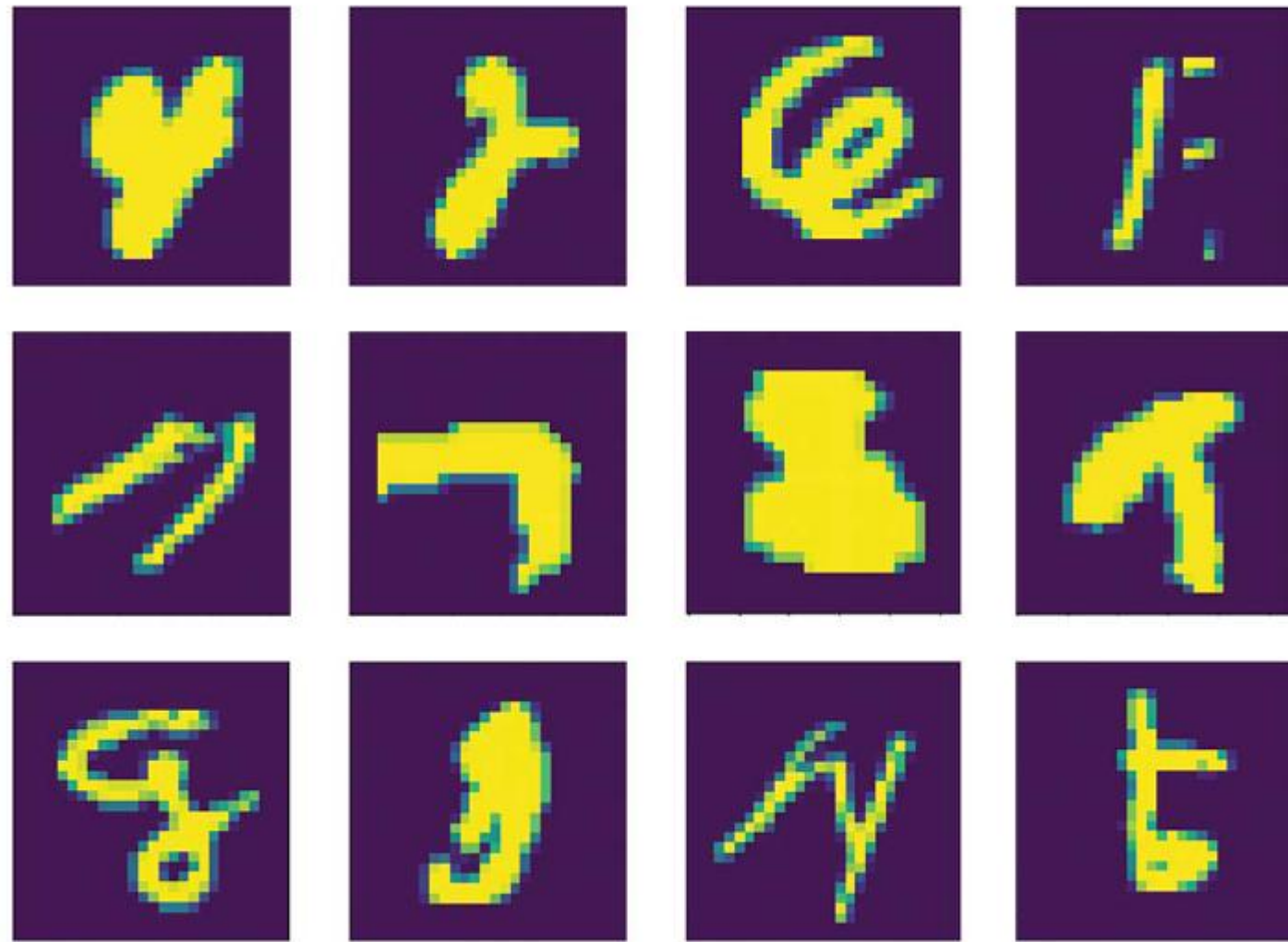


Figure 5.2 Some pretty weird MNIST training samples

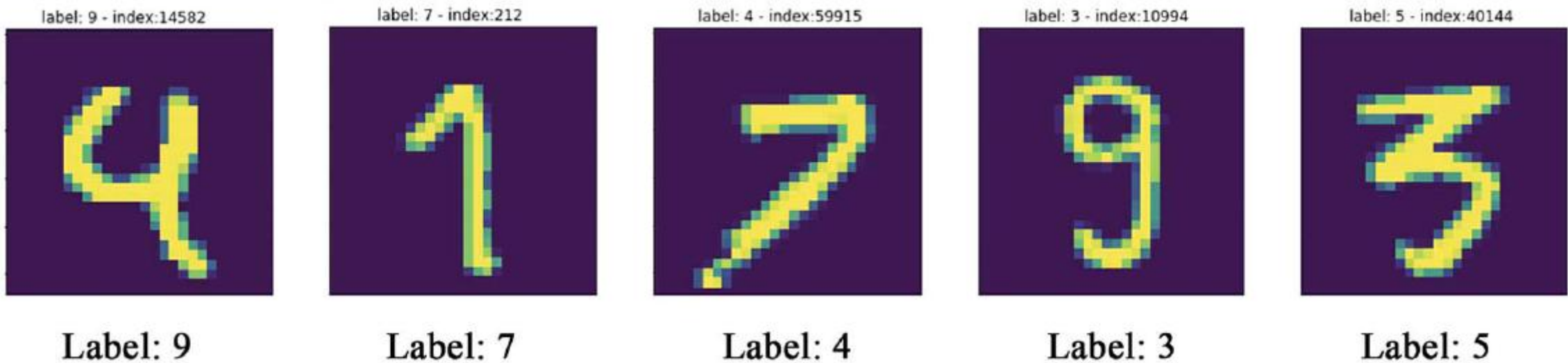


Figure 5.3 Mislabeled MNIST training samples

Vì sao mislabeled samples nguy hiểm?

- Gắn nhãn sai → model bị “dạy sai”
- Ví dụ ảnh là số 4 nhưng label là 9: Model nhìn thấy hình số 4 → bị bắt học rằng đây là số 9.
- Nếu có nhiều mẫu sai như vậy, model có thể:
 - Học nhầm pattern.
 - Training khó ổn định hơn.
 - Dễ overfit vào dữ liệu sai.
 - Validation/test performance giảm.
 - Khó đạt độ chính xác cao dù model đủ mạnh.

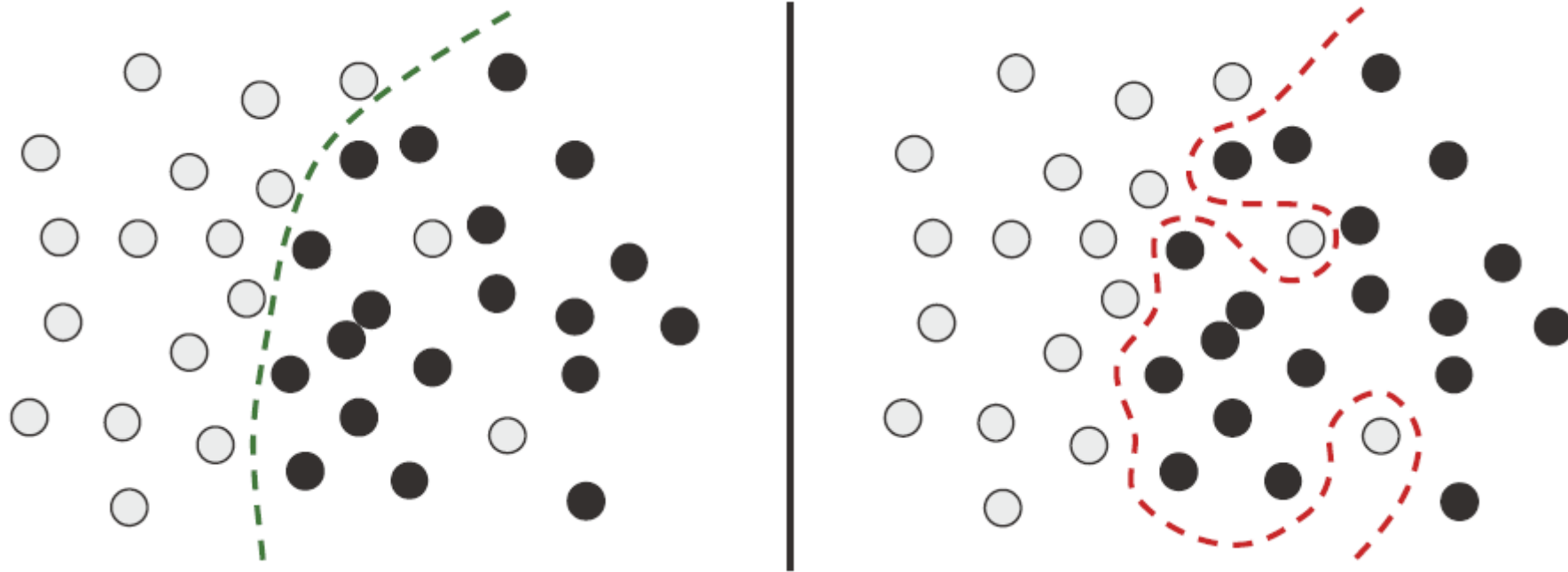


Figure 5.4 Dealing with outliers: robust fit vs. overfitting

Trường hợp	Đặc điểm	Khả năng khái quát hóa
Robust fit	Ranh giới đơn giản, mượt, bỏ qua một số outliers	Thường tốt hơn
Overfitting	Ranh giới phức tạp, chạy theo từng điểm ngoại lai	Thường kém hơn

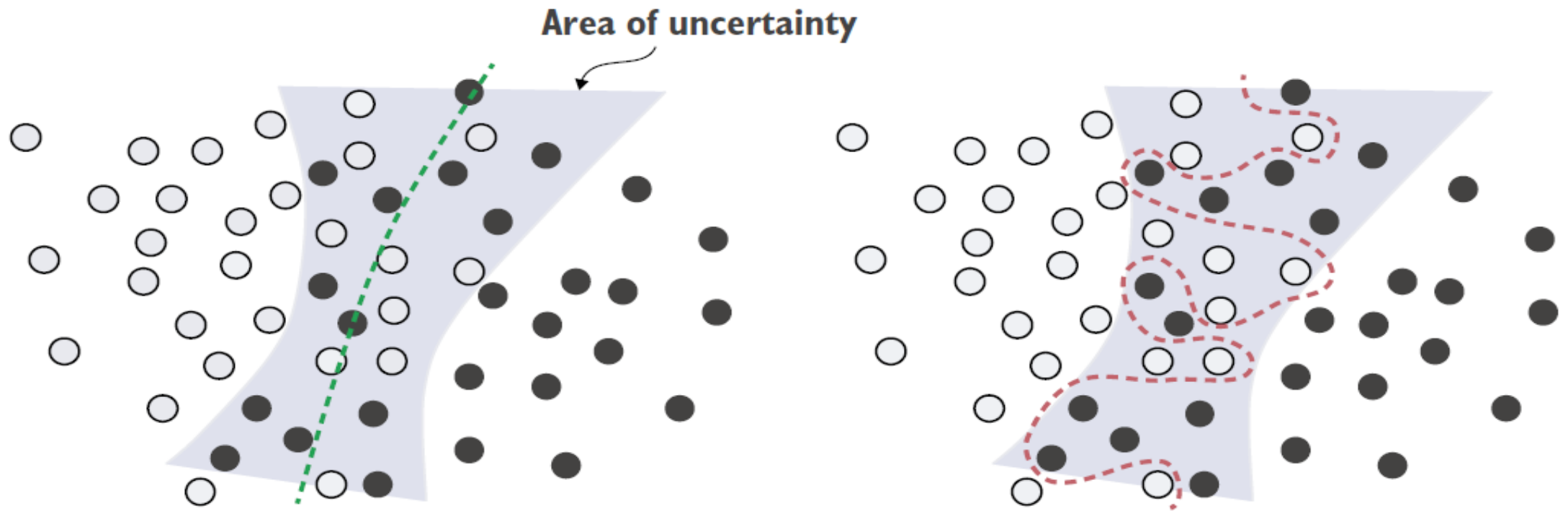


Figure 5.5 Robust fit vs. overfitting giving an ambiguous area of the feature space

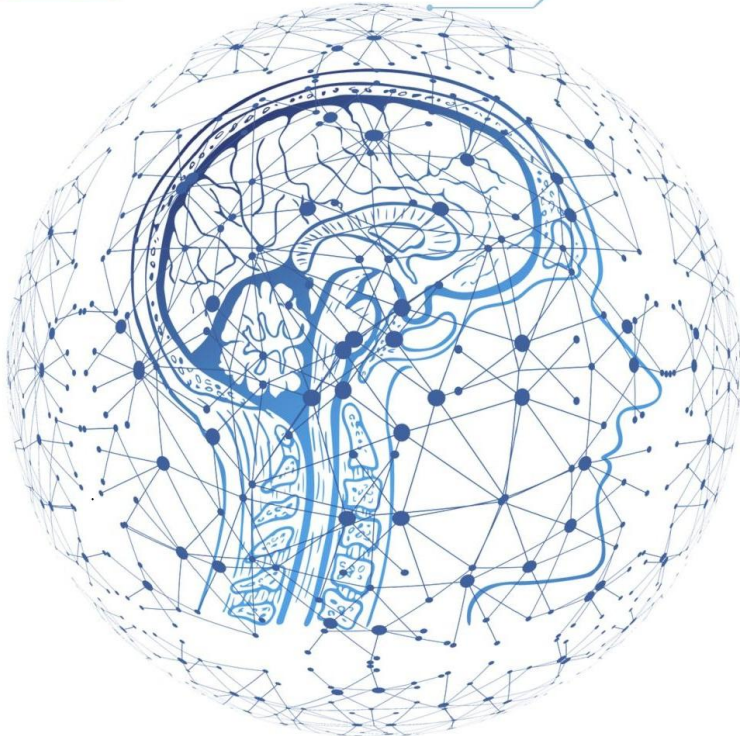
Mô hình	Đặc điểm	Kết quả
Robust fit	Ranh giới đơn giản, mượt, không chạy theo từng điểm	Generalize tốt hơn
Overfitting	Ranh giới phức tạp, uốn lượn theo training data	Training tốt nhưng test có thể kém

Bản chất của khái quát hóa

- Deep learning có thể khái quát hóa tốt dù mô hình có rất nhiều tham số
- Thực tế: generalization còn phụ thuộc vào:
 - dữ liệu
 - bài toán
 - cách huấn luyện
 - regularization - điều chuẩn

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Đánh giá các mô hình học máy

Evaluating machine learning models

Đánh giá các mô hình học máy

- Đánh giá mô hình là bước bắt buộc
- Không đánh giá đúng → mọi kết luận đều dễ sai
- Ba tập dữ liệu thường dùng:
 - training
 - validation
 - test

Training Set, Validation Set, Test Set

■ **Training set:**

- là tập dữ liệu dùng để **huấn luyện model**.
- dùng để học tham số: weights, bias,

■ **Validation set:**

- dùng để chọn mô hình / chọn hyperparameters
- dùng để kiểm tra model trong quá trình phát triển

■ **Test set:**

- dùng để đánh giá khách quan xem model có hoạt động tốt trên dữ liệu mới hay không sau khi huấn luyện

Validation Set

- **Validation set** không dùng để model học trực tiếp weights và bias, mà dùng để **kiểm tra model trong quá trình phát triển**.
- Ta dùng validation set để quyết định:
 - chọn model nào tốt hơn
 - chọn số layer bao nhiêu
 - chọn learning rate bao nhiêu
 - chọn batch size bao nhiêu
 - chọn số epoch dừng ở đâu
 - có cần dropout, L2 regularization (là kỹ thuật giảm overfitting) hay không

Validation Set

- Ví dụ: Ta thử 3 model:

Model	Accuracy trên validation
Model A	82%
Model B	88%
Model C	85%

- Ta có thể chọn **Model B** vì nó hoạt động tốt nhất trên validation set.

Vì sao không đánh giá trên tập train?

- Vì mô hình đã nhìn thấy dữ liệu đó
- Kết quả sẽ quá lạc quan
- Không phản ánh năng lực trên dữ liệu mới

Các chiến lược đánh giá phổ biến

- Hold-out validation: Đánh giá bằng tập validation tách riêng
- K-fold cross-validation: Đánh giá chéo K lần
- Iterated K-fold validation: Đánh giá K-fold lặp lại nhiều lần

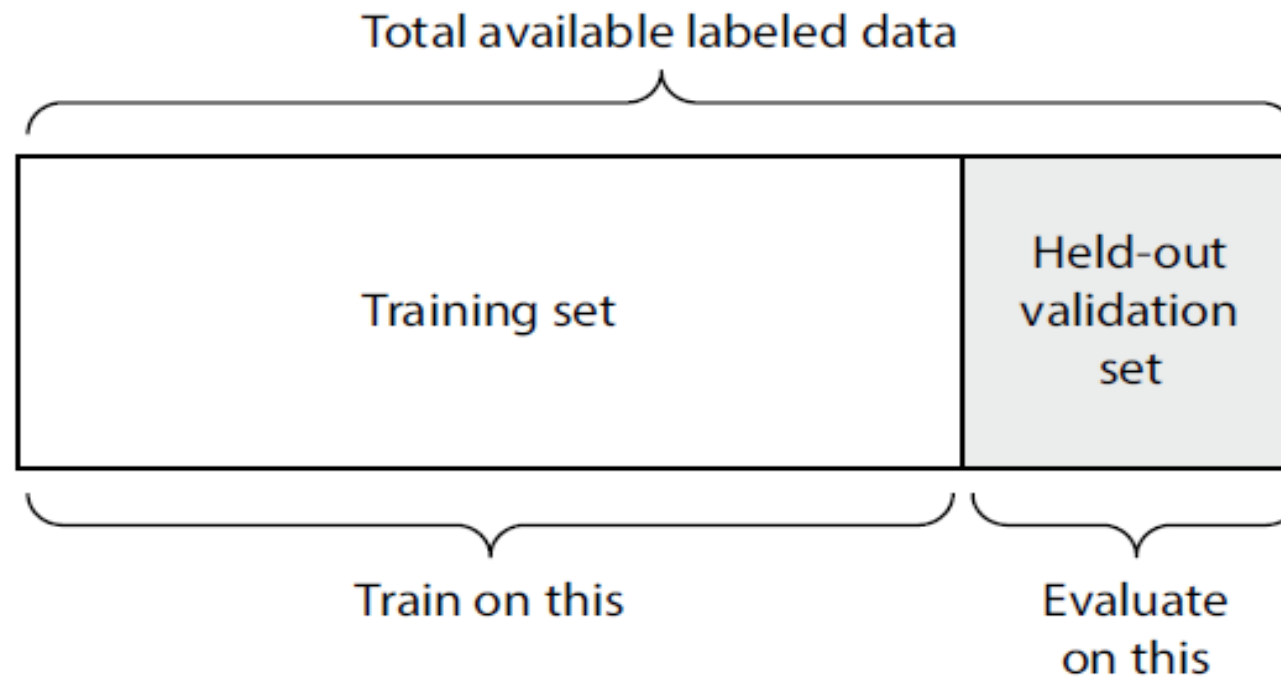


Figure 5.12 Simple holdout validation split

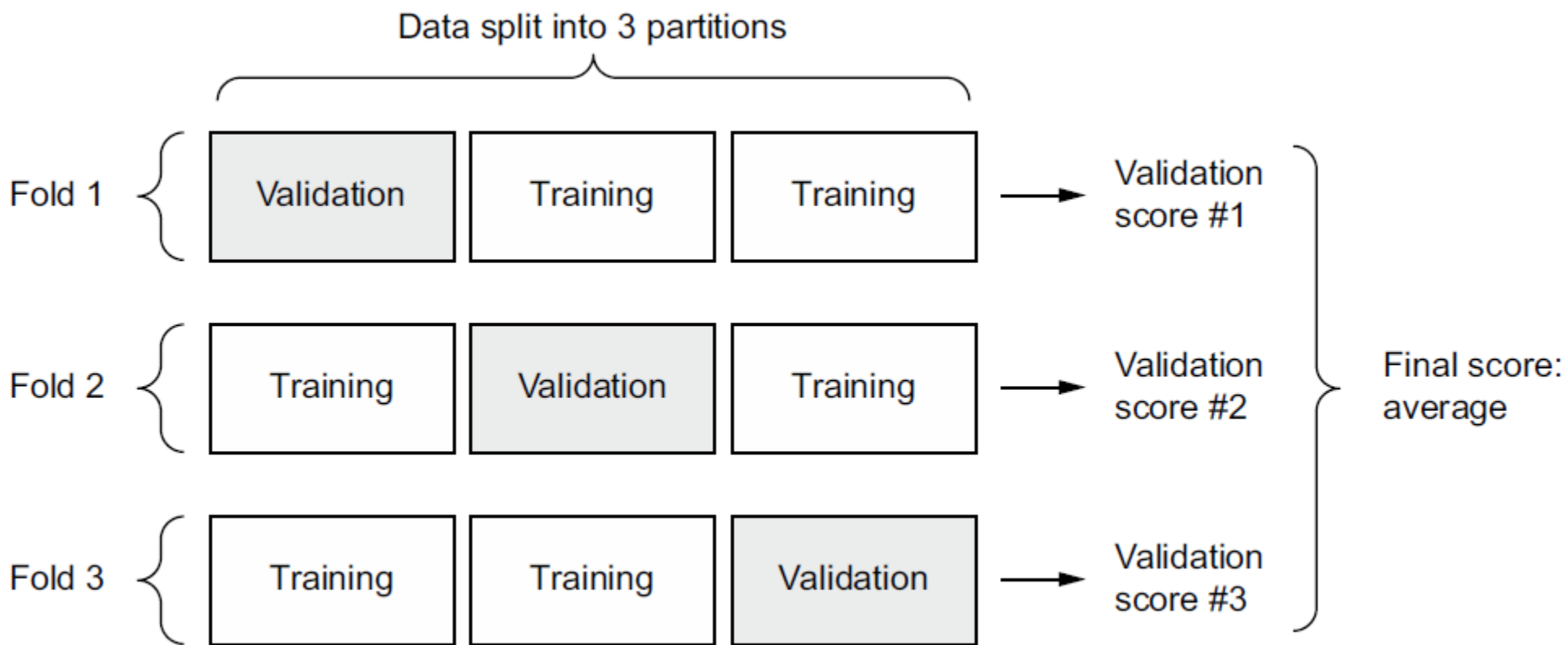


Figure 5.13 K-fold cross-validation with K=3

So sánh các chiến lược đánh giá phổ biến

Phương pháp	Cách làm	Ưu điểm	Nhược điểm	Khi dùng
Hold-out validation	Chia dữ liệu một lần	Đơn giản, nhanh	Phụ thuộc cách chia	Dataset lớn
K-fold cross-validation	Chia K phần, train K lần	Đánh giá ổn định hơn	Tốn thời gian hơn	Dataset nhỏ/vừa
Iterated K-fold validation	Lặp lại K-fold nhiều lần	Rất ổn định	Rất tốn compute	Dataset rất nhỏ hoặc cần đánh giá kỹ

Common-sense baseline

Mốc so sánh đơn giản theo trực giác

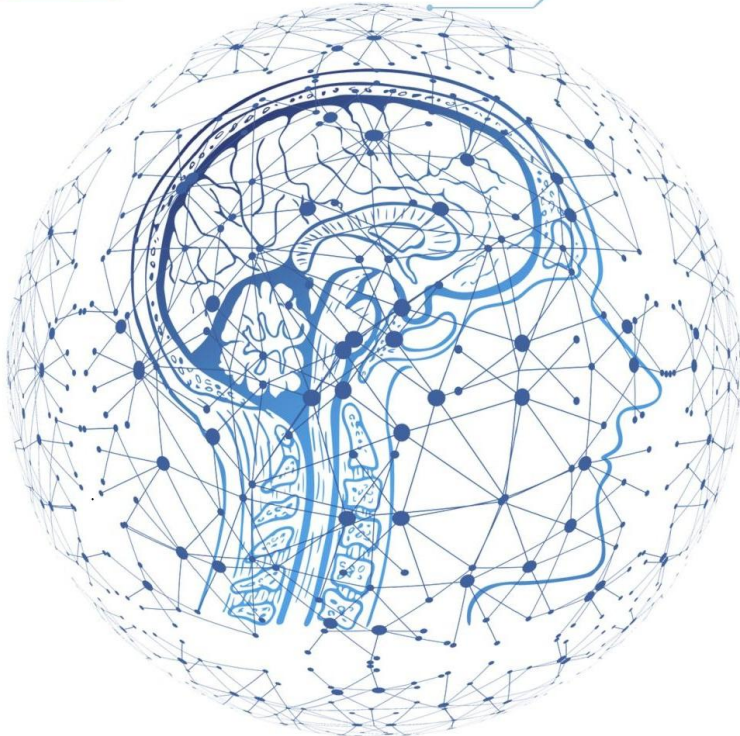
- Common-sense baseline là mốc so sánh đơn giản, thường dựa trên trực giác hoặc quy tắc rất cơ bản, dùng làm mốc tối thiểu để đánh giá model.
- Nếu model phức tạp mà **không vượt qua được mốc này**, thì model chưa thật sự có giá trị.
- Common-sense baseline thông dụng:
 - **Dự đoán class phổ biến nhất.** Ví dụ: nếu 90% email là “không spam”, baseline luôn dự đoán “không spam”.
 - **Dự đoán ngẫu nhiên.** Ví dụ: với bài toán phân loại positive/negative cân bằng 50–50, baseline đoán ngẫu nhiên một trong hai nhãn.
 - **Dự đoán giá trị trung bình/trung vị.** Ví dụ: với bài toán dự đoán giá nhà, baseline luôn dự đoán bằng giá nhà trung bình
 - **Dự đoán theo giá trị gần nhất trước đó.** Ví dụ: dự đoán thời tiết ngày mai giống hôm nay, hoặc doanh thu tháng sau giống tháng này.
 - **Dự đoán theo một rule đơn giản.** Ví dụ: nếu khách hàng không mua hàng trong 6 tháng thì dự đoán có khả năng rời bỏ.

Những điều cần nhớ khi đánh giá mô hình

- Tách dữ liệu cẩn thận
- Tránh **data leakage (rò rỉ dữ liệu)**
- Validation không được “nhiễm” thông tin từ test
- Đừng ra quyết định dựa trên một chỉ số duy nhất nếu bài toán phức tạp. Ví dụ: không nên chỉ nhìn một metric như accuracy rồi kết luận model tốt hay xấu.

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Cải thiện
độ phù hợp của mô hình
Improving model fit

Cải thiện độ phù hợp của mô hình

- Nếu mô hình còn underfitting, cần cải thiện việc huấn luyện
- Ba hướng chính:
 - tuning gradient descent parameters: tinh chỉnh các tham số của thuật toán gradient gradient
 - better architecture priors: lựa chọn kiến trúc mô hình phù hợp hơn
 - increasing model capacity: tăng khả năng học của mô hình

Tinh chỉnh tham số của thuật toán gradient gradient

- Điều chỉnh:
 - learning rate
 - batch size
 - số epochs
 - optimizer
- Các tham số này ảnh hưởng mạnh đến quá trình học

Lựa chọn kiến trúc mô hình phù hợp hơn

- Chọn kiến trúc phù hợp với loại dữ liệu
- Ví dụ:
 - ảnh $\rightarrow$ CNN
 - chuỗi thời gian $\rightarrow$ RNN / 1D Conv
 - văn bản $\rightarrow$ Embedding / sequence models
- Kiến trúc phù hợp giúp mô hình học dễ hơn

Tăng khả năng học của mô hình

- Nếu validation loss giảm rồi dừng lại ở mức chưa tốt, **model có thể đang quá yếu và cần tăng capacity.**
- Mục tiêu: **giảm underfitting**
- Nhưng tăng quá mức có thể dẫn đến overfitting

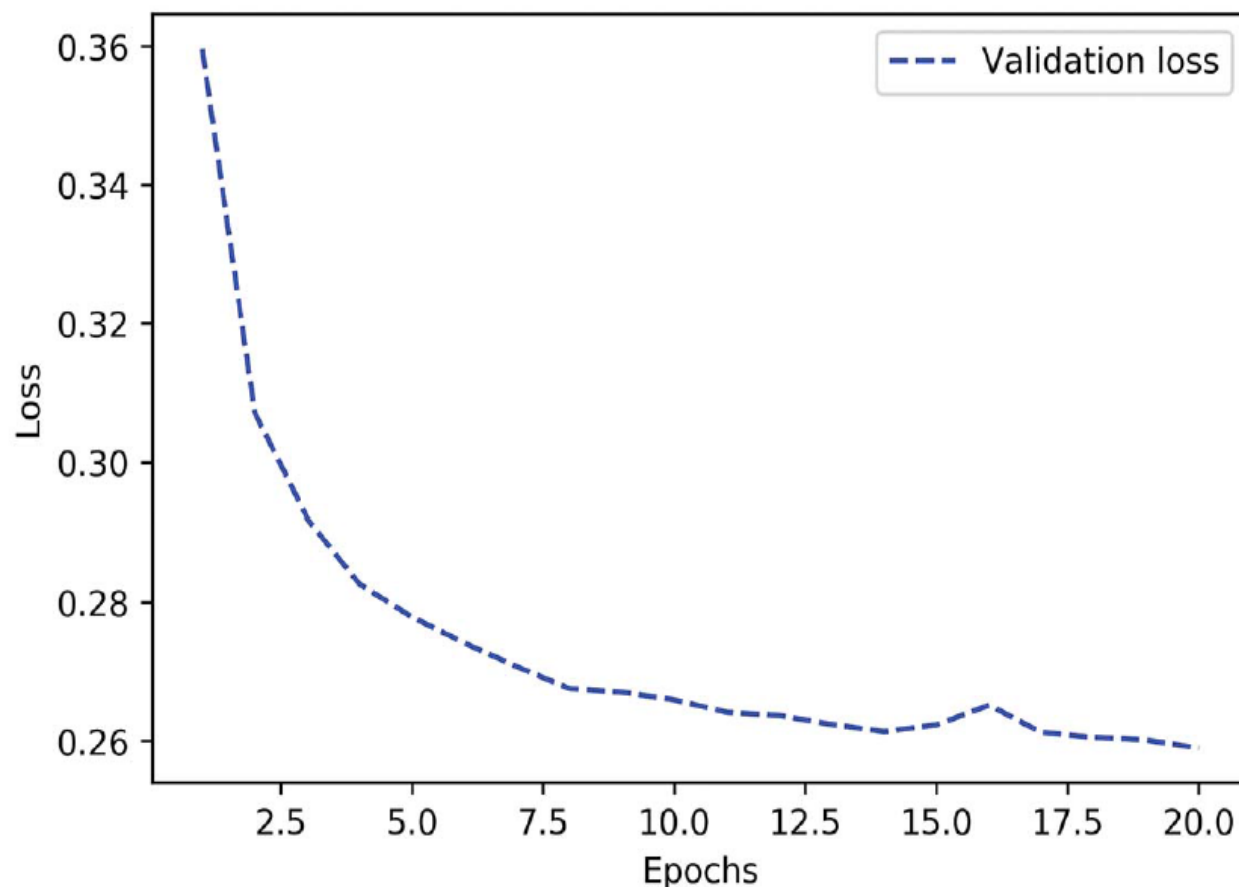


Figure 5.14 Effect of insufficient model capacity on loss curves

Cách tăng khả năng học của mô hình

- **Tăng số layer:** Cho model nhiều tầng xử lý hơn.
- **Tăng số units/neurons mỗi layer:** Cho model nhiều khả năng biểu diễn hơn.
- **Dùng kiến trúc phù hợp hơn:**
 - Ảnh: dùng CNN/ConvNet.
 - Chuỗi thời gian: dùng RNN, LSTM, Transformer hoặc Conv1D.
 - Văn bản: dùng embedding, Transformer, v.v.
- **Train lâu hơn,** nếu loss vẫn còn đang giảm. Nhưng trong hình này loss đã gần như chững lại, nên chỉ train lâu hơn có thể không đủ.
- **Cải thiện feature engineering:** Cung cấp đặc trưng tốt hơn để model dễ học hơn.

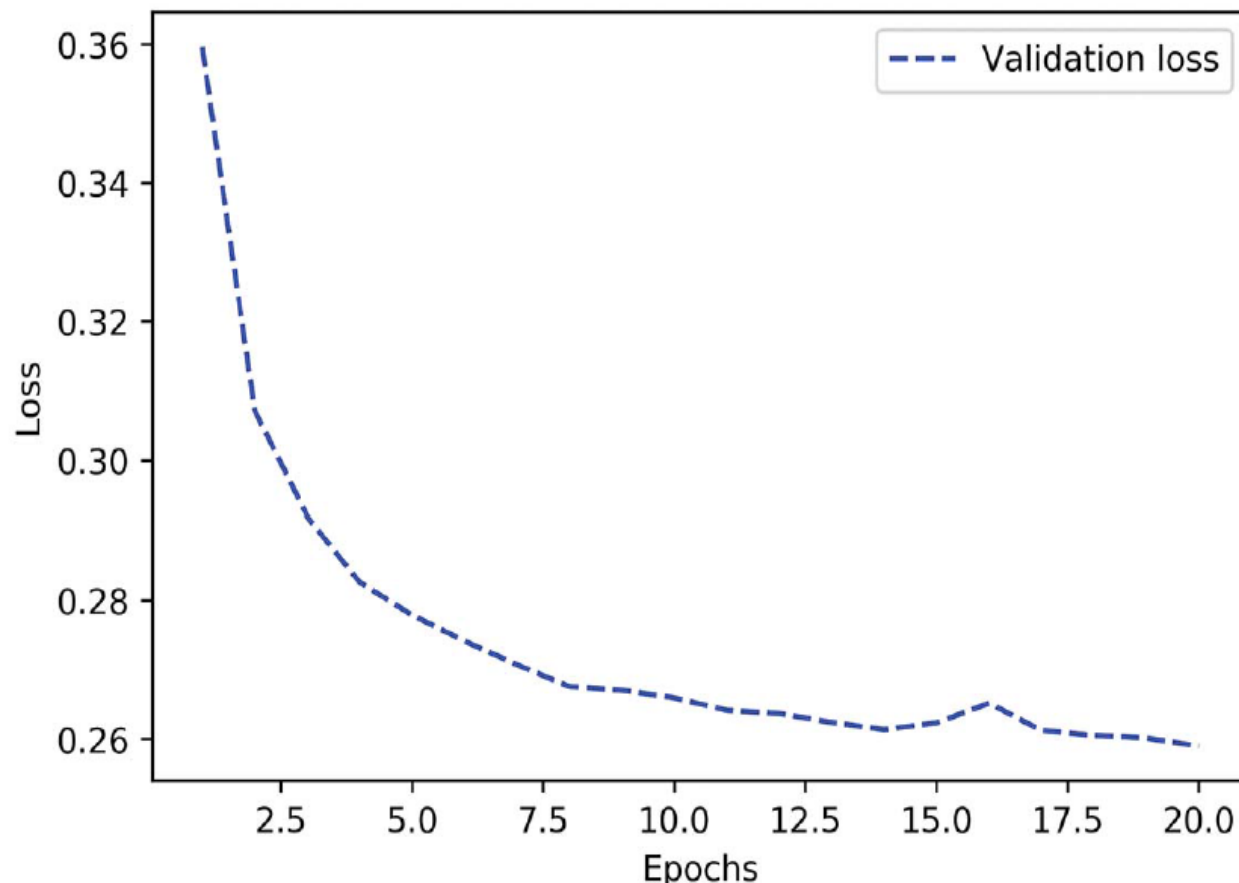


Figure 5.14 Effect of insufficient model capacity on loss curves

Model có capacity phù hợp

Quan sát validation loss:

- Ở giai đoạn đầu:
 - Epoch 1 → validation loss khoảng 0.18
 - Epoch 6–8 → validation loss giảm xuống khoảng 0.09–0.10
 - Điều này cho thấy model đang học tốt và khả năng generalization được cải thiện.
- Nhưng sau đó:
 - Từ khoảng epoch 9 trở đi → validation loss bắt đầu tăng dần
 - Nghĩa là model tiếp tục học training data, nhưng hiệu quả trên validation data lại xấu đi.

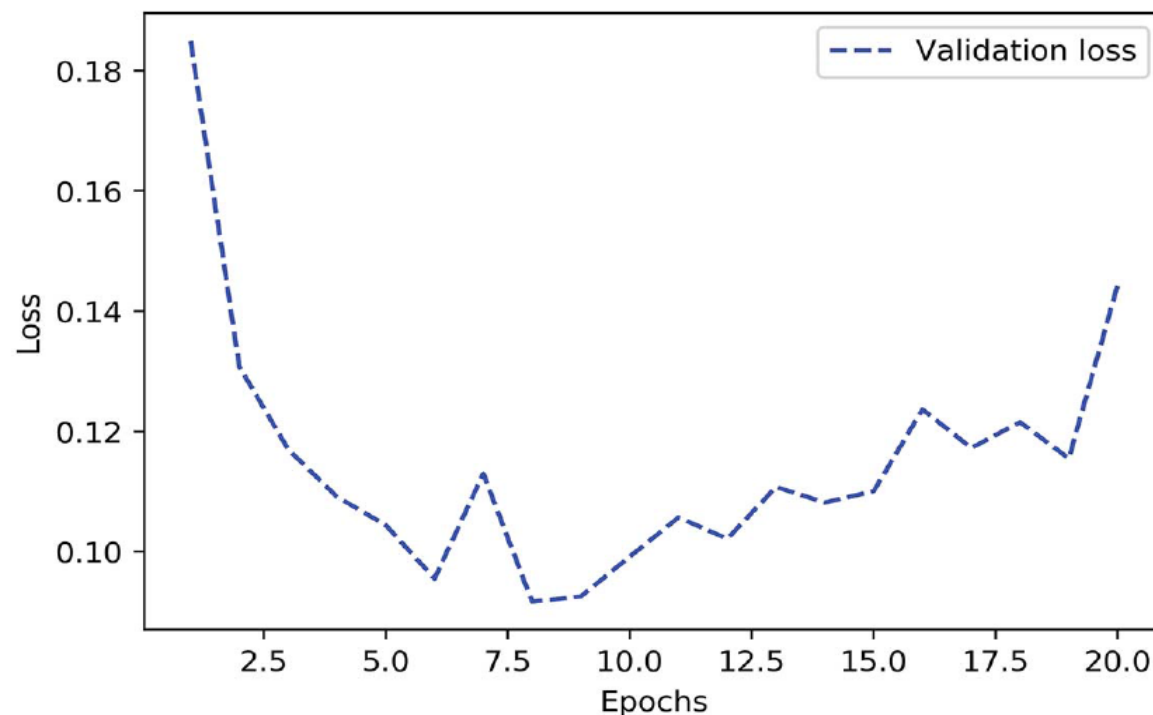


Figure 5.15 Validation loss for a model with appropriate capacity

Model có capacity phù hợp

Ý nghĩa của đồ thị:

- Đây là biểu hiện rất điển hình của quá trình:
 - Ban đầu: model học pattern tổng quát
 - Sau đó: model bắt đầu overfit training data
- Ở những epoch đầu, model học các quy luật thật sự có ích. Vì vậy validation loss giảm.
- Sau một số epoch, model bắt đầu học thêm các chi tiết riêng của training set, bao gồm:
 - nhiễu
 - ngoại lệ
 - pattern không đại diện
 - chi tiết quá cụ thể

→ Khi đó validation loss tăng lên.

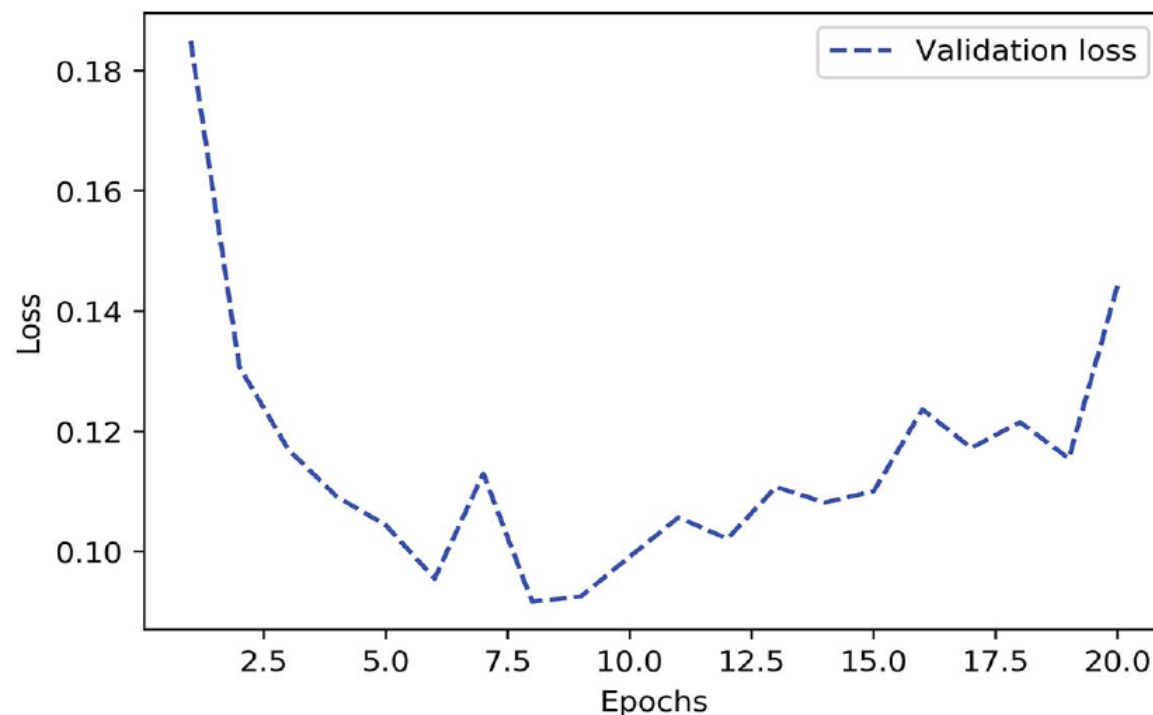


Figure 5.15 Validation loss for a model with appropriate capacity

Model có capacity phù hợp

Vì sao gọi là model có capacity phù hợp?

- Model này đủ mạnh để:
 - học tốt pattern trong dữ liệu
 - làm validation loss giảm đáng kể
 - đạt được mức loss thấp hơn model quá yếu
- Nhưng nó không quá yếu như hình trước, vì loss không bị chững lại sớm ở mức cao.
- Tuy nhiên, nếu train quá lâu, model vẫn có thể overfit. **Đây là hiện tượng bình thường với model có capacity đủ tốt.**

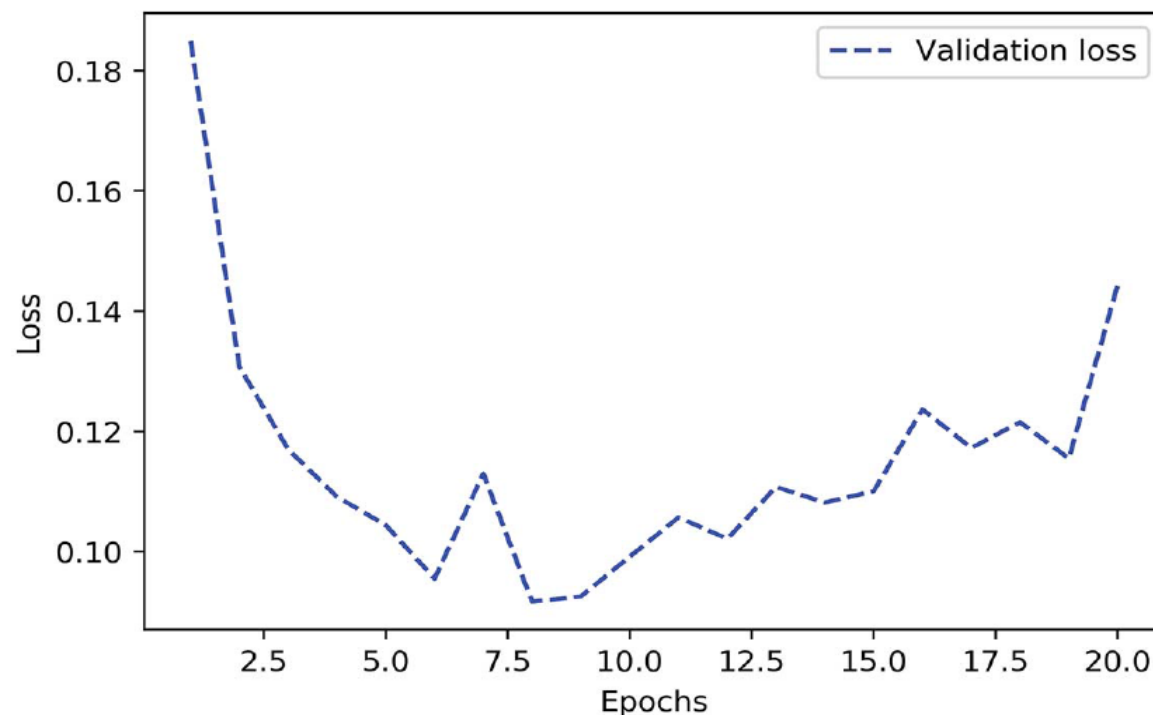


Figure 5.15 Validation loss for a model with appropriate capacity

Model có capacity phù hợp

Nên chọn model ở epoch nào?

- Ta nên chọn model tại điểm **validation loss thấp nhất**, khoảng: Epoch 8 hoặc 9
- Vì sau thời điểm đó, validation loss bắt đầu tăng, nghĩa là model generalize kém hơn.
- Đây chính là ý tưởng của:
 - **Early stopping / Dừng sớm:** Tức là dừng training khi validation performance không còn cải thiện.

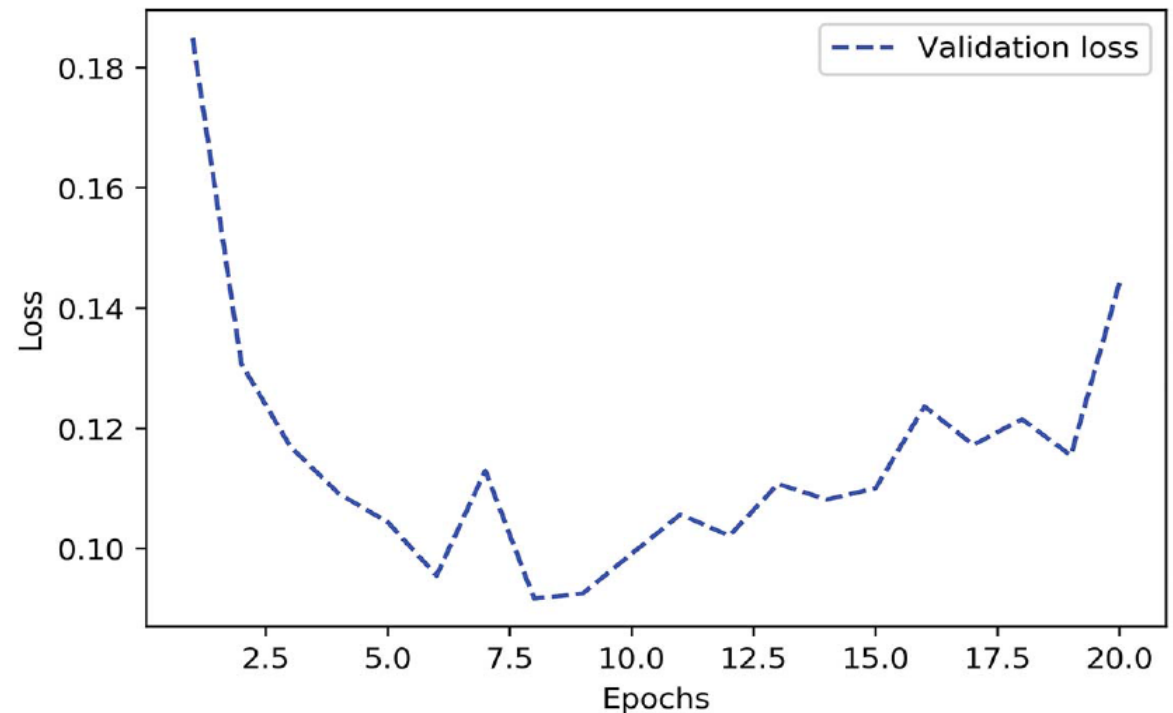
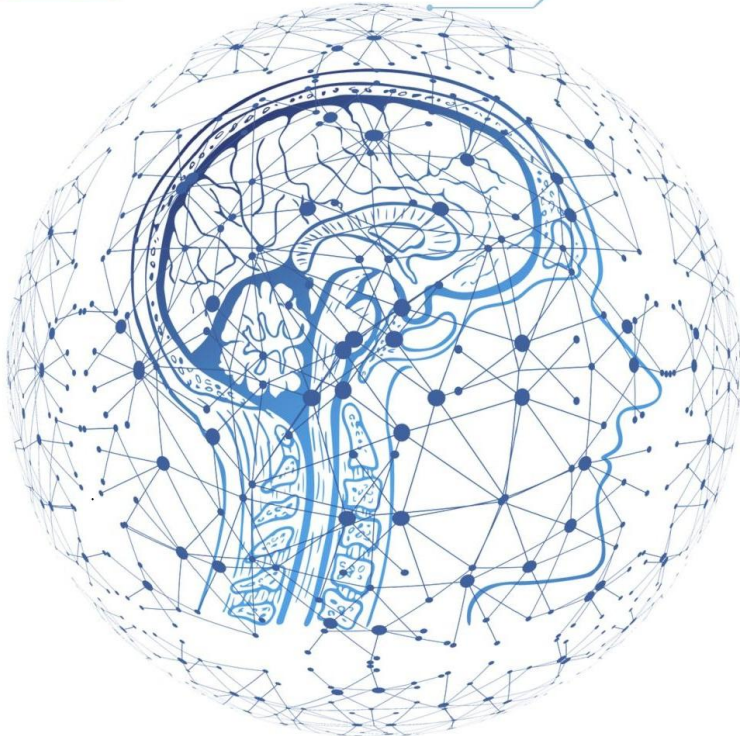


Figure 5.15 Validation loss for a model with appropriate capacity

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



Cải thiện
khả năng khái quát hóa
Improving generalization

Cải thiện khả năng khái quát hóa

- Generalization có thể cải thiện bằng:
 - dataset curation: chọn lọc và quản lý chất lượng bộ dữ liệu
 - feature engineering: thiết kế đặc trưng
 - early stopping: dừng sớm
 - regularization: điều chuẩn

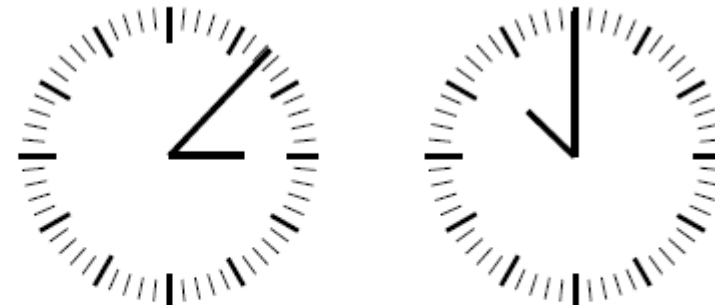
Chọn lọc và quản lý chất lượng bộ dữ liệu

- Có đủ dữ liệu
- Giảm lỗi gán nhãn
- Làm sạch dữ liệu
- Xử lý missing values
- Chọn lọc feature nếu cần

Thiết kế đặc trưng

- Dùng hiểu biết về bài toán để biến đổi dữ liệu đầu vào
- Mục tiêu:
 - làm bài toán đơn giản hơn
 - giúp mô hình học dễ hơn
- Không phải lúc nào cũng phải để mô hình tự học tất cả

Raw data:
pixel grid



Better
features:
clock hands'
coordinates

{x1: 0.7,
y1: 0.7}
{x2: 0.5,
y2: 0.0}

{x1: 0.0,
y2: 1.0}
{x2: -0.38,
y2: 0.32}

Even better
features:
angles of
clock hands

theta1: 45
theta2: 0

theta1: 90
theta2: 140

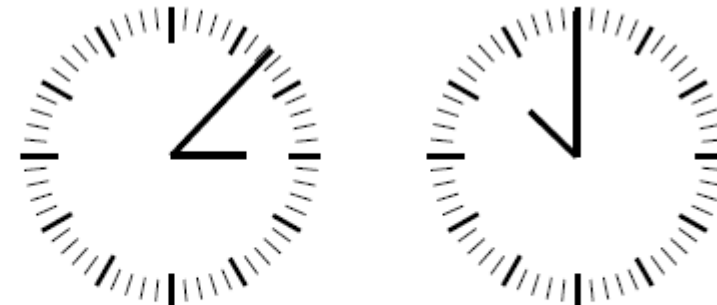
Figure 5.16 Feature engineering for reading the time on a clock

Thiết kế đặc trưng

■ Dữ liệu thô: lưới pixel

- Ở hàng đầu tiên, đồng hồ được biểu diễn dưới dạng **ảnh thô**, tức là một lưới pixel.
- Máy tính nhìn ảnh không giống con người. Nó không thấy ngay “kim giờ”, “kim phút”, “góc 45 độ”. Nó chỉ thấy rất nhiều điểm ảnh sáng/tối.
- Ví dụ:
 - Raw data = ma trận pixel của ảnh đồng hồ
- Vấn đề là: từ pixel thô, model phải tự học ra đâu là kim đồng hồ, đâu là mặt đồng hồ, kim đang chỉ hướng nào. **Việc này khó hơn.**

Raw data:
pixel grid



Better
features:
clock hands'
coordinates

{x1: 0.7,
y1: 0.7}
{x2: 0.5,
y2: 0.0}

{x1: 0.0,
y2: 1.0}
{x2: -0.38,
y2: 0.32}

Even better
features:
angles of
clock hands

theta1: 45
theta2: 0

theta1: 90
theta2: 140

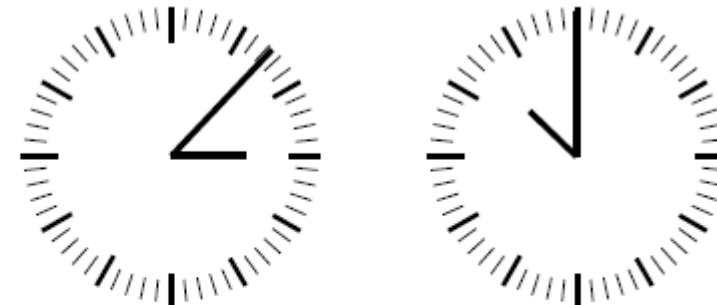
Figure 5.16 Feature engineering for reading the time on a clock

Thiết kế đặc trưng

■ Đặc trưng tốt hơn: tọa độ kim đồng hồ

- Ở hàng thứ hai, thay vì dùng toàn bộ ảnh pixel, ta trích ra tọa độ hai kim đồng hồ: $\{x_1, y_1\}, \{x_2, y_2\}$
- Cách biểu diễn này tốt hơn pixel thô vì model không cần tự tìm kim đồng hồ trong ảnh nữa. Ta đã giúp model bằng cách đưa thẳng thông tin quan trọng: **vị trí của hai kim.**
- Nhưng vẫn còn một bước suy luận: **từ tọa độ, model phải học tiếp rằng tọa độ đó tương ứng với góc bao nhiêu.**

Raw data:
pixel grid



Better
features:
clock hands'
coordinates

$\{x_1: 0.7,$
 $y_1: 0.7\}$
 $\{x_2: 0.5,$
 $y_2: 0.0\}$

$\{x_1: 0.0,$
 $y_2: 1.0\}$
 $\{x_2: -0.38,$
 $y_2: 0.32\}$

Even better
features:
angles of
clock hands

theta1: 45
theta2: 0

theta1: 90
theta2: 140

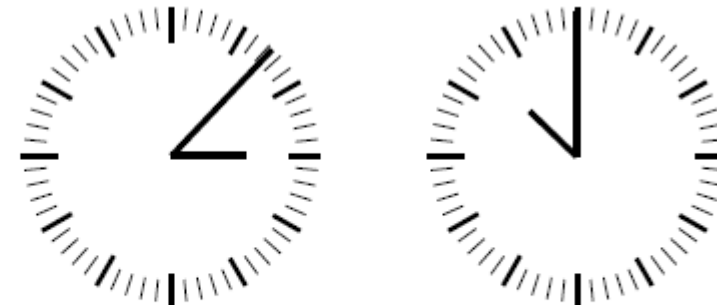
Figure 5.16 Feature engineering for reading the time on a clock

Thiết kế đặc trưng

▪ Đặc trưng tốt hơn nữa: góc của kim đồng hồ

- Ở hàng thứ ba, dữ liệu được biểu diễn trực tiếp bằng góc của hai kim.
- Đây là đặc trưng tốt hơn nữa, vì trong bài toán đọc giờ, điều quan trọng nhất không phải pixel hay tọa độ, mà là **góc quay của kim đồng hồ**.
- Khi có góc rồi, model dễ học hơn nhiều.

Raw data:
pixel grid



Better
features:
clock hands'
coordinates

{x1: 0.7,
y1: 0.7}
{x2: 0.5,
y2: 0.0}

{x1: 0.0,
y2: 1.0}
{x2: -0.38,
y2: 0.32}

Even better
features:
angles of
clock hands

theta1: 45
theta2: 0

theta1: 90
theta2: 140

Figure 5.16 Feature engineering for reading the time on a clock

Dừng sớm khi train

- Theo dõi validation metric trong quá trình train
- Dừng sớm khi validation không còn cải thiện
- Giúp tránh train quá lâu và overfit
- Ví dụ: Trong Keras: EarlyStopping callback sẽ dừng khi validation ngừng cải thiện.

Điều chuẩn

- Regularization: chủ động làm mô hình khó học thuộc dữ liệu train hơn
- Mục tiêu:
 - giảm overfitting
 - tăng khả năng khái quát hóa
- Các cách regularization thường gặp:
 - Giảm kích thước mạng
 - L1 / L2 regularization
 - Dropout
 - Data augmentation (trong các bài toán phù hợp)

Giảm kích thước model

- **Giảm kích thước model là một cách regularization**, vì nó giới hạn khả năng model học thuộc training data
- Model quá lớn → dễ ghi nhớ training data → overfit nhanh.
- Model nhỏ hơn → ít tham số hơn → overfit chậm hơn → validation loss tăng chậm hơn.

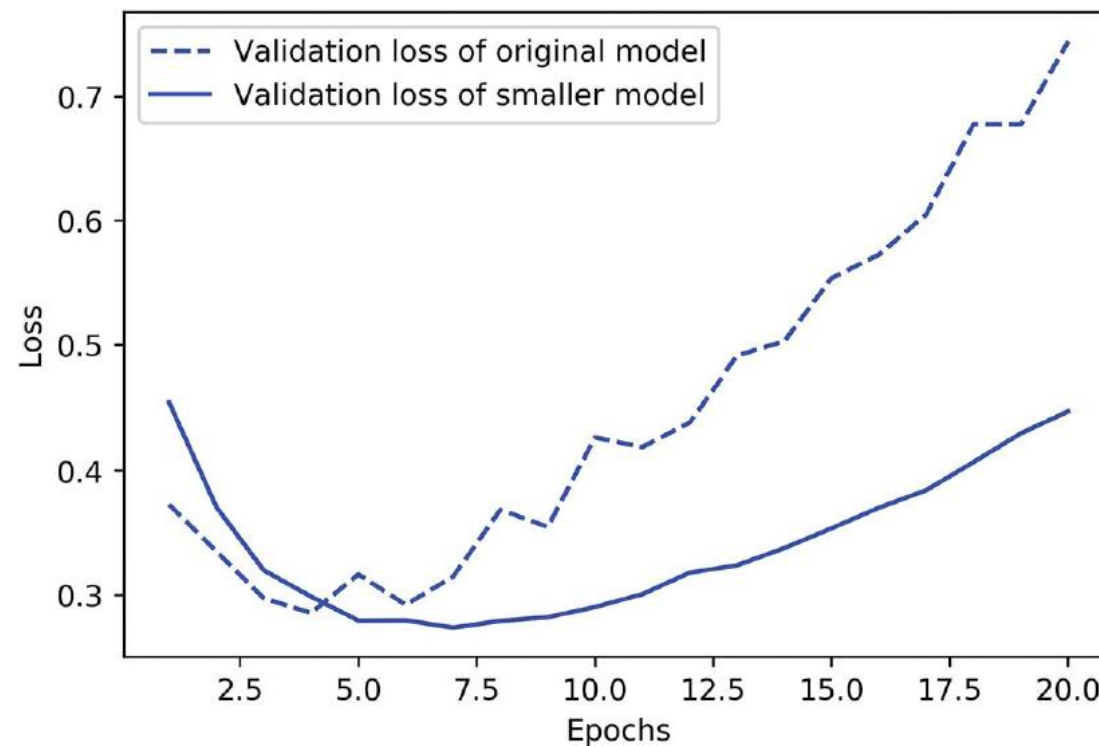


Figure 5.17 Original model vs. smaller model on IMDB review classification

Tăng kích thước model

- **Tăng kích thước model không phải lúc nào cũng tốt.**
- Model quá lớn có thể học thuộc training data và làm validation performance kém đi.
- Model lớn hơn → nhiều tham số hơn → dễ ghi nhớ training data hơn → overfit nhanh hơn.

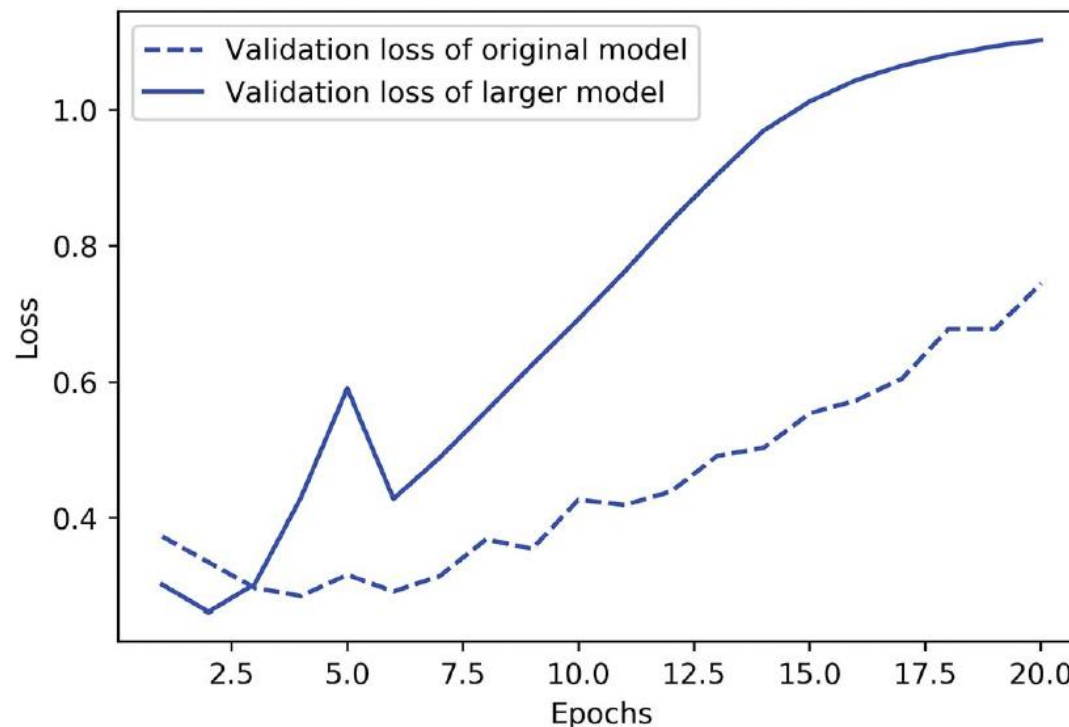


Figure 5.18 Original model vs. much larger model on IMDB review classification

L2 weight regularization/Điều chuẩn trọng số L2

- Model ban đầu (không áp dụng regularization) bắt đầu overfit sau vài epoch.
- **L2-regularized model** ổn định hơn nhiều. Validation loss cũng có tăng, nhưng:
 - tăng chậm hơn,
 - không tăng vọt như original model,
 - ít biểu hiện overfitting nghiêm trọng hơn.
- Điều này cho thấy L2 regularization làm model **resistant to overfitting / kháng overfitting tốt hơn** so với model ban đầu, dù hai model có cùng số lượng parameters.

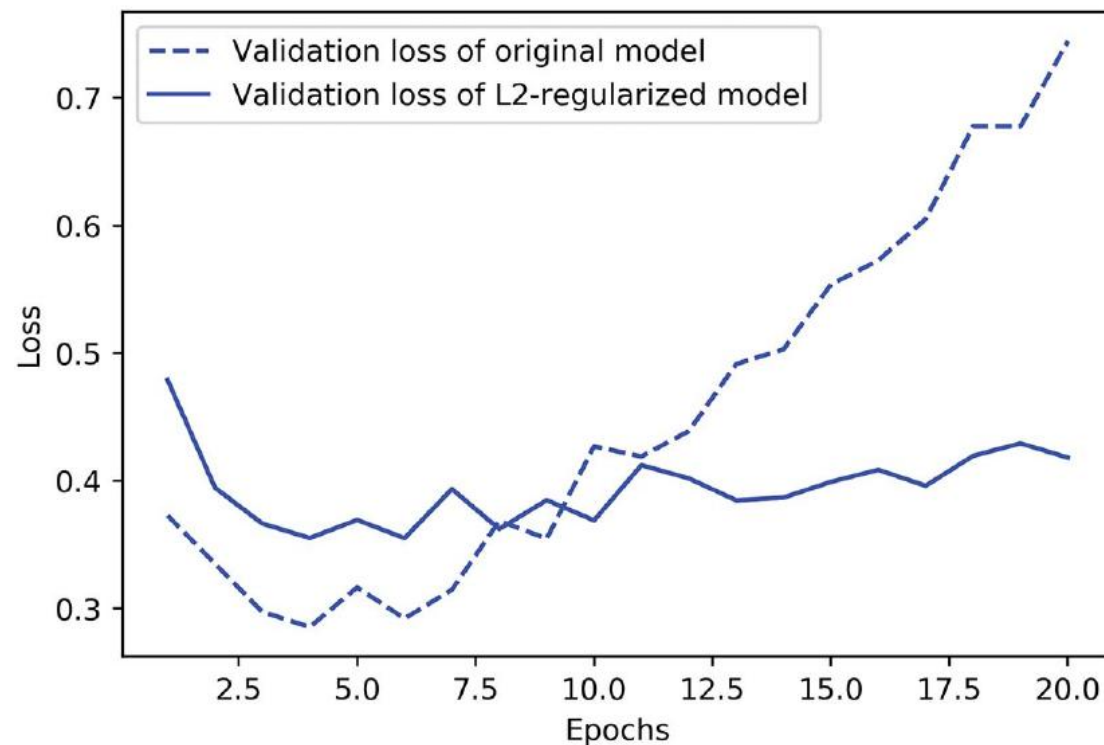


Figure 5.19 Effect of L2 weight regularization on validation loss

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

- **Dropout** là kỹ thuật regularization giúp giảm overfitting bằng cách **ngẫu nhiên đặt một phần activation về 0 trong lúc training.**
- Nhờ vậy, model không phụ thuộc quá nhiều vào một vài neuron cụ thể và học được biểu diễn tổng quát hơn.

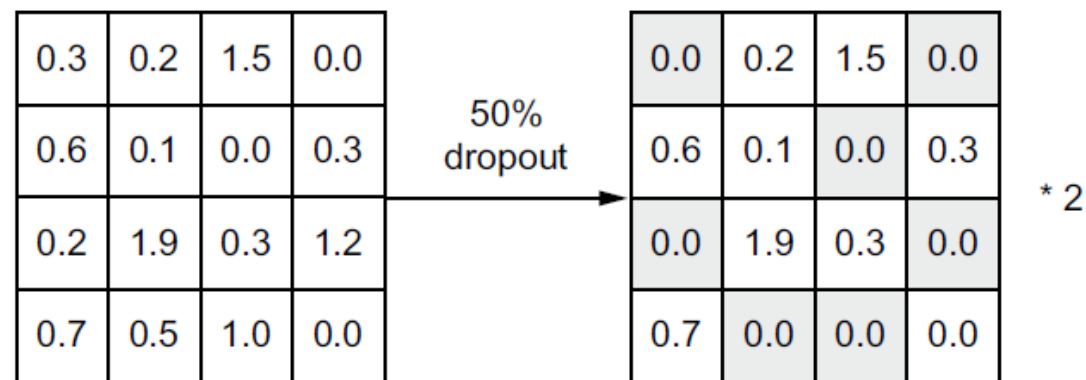


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

1. Ma trận bên trái là gì?

- Là các **activation values** / **giá trị kích hoạt** của một layer trong neural network.

2. "50% dropout" nghĩa là gì?

- **50% dropout** nghĩa là trong lúc training, model sẽ **ngẫu nhiên tắt 50% giá trị activation**.
- "Tắt" ở đây nghĩa là đổi giá trị đó thành **0**.

3. Ma trận bên phải thể hiện điều gì?

- Sau khi áp dụng dropout, một số activation bị đặt thành 0.
- Nghĩa là trong lần training này, model chỉ được dùng một phần các activation.
- Ở batch khác, dropout sẽ tắt các vị trí khác.

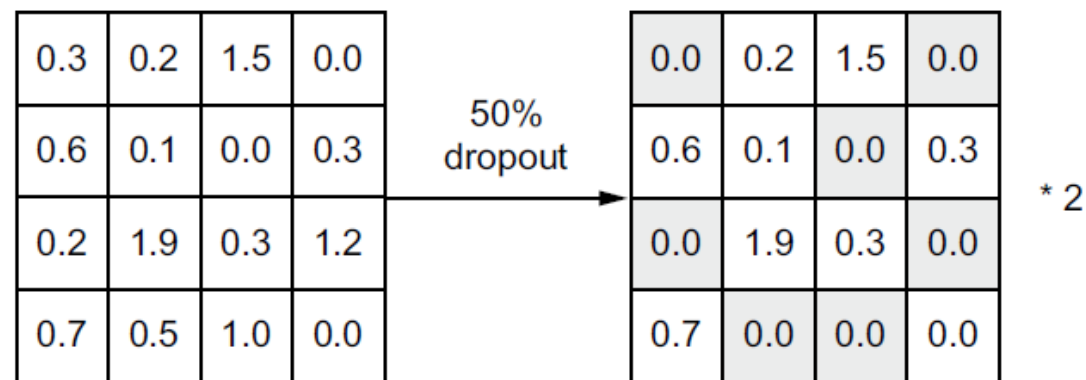


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

4. Vì sao có nhân *2?

- Vì dropout rate = 50%, tức là chỉ còn trung bình **một nửa activation được giữ lại**.
- Nếu ta tắt 50% activation mà không điều chỉnh gì, tổng tín hiệu đi qua layer sẽ bị giảm đi khoảng một nửa.
- Do đó, các activation còn lại được **rescale / nhân bù** lên: Giữ lại 50% → nhân các giá trị còn lại với 2
- Tổng quát:
 - dropout rate = 0.5 → keep rate = 0.5 → nhân với $1 / 0.5 = 2$
 - Nếu dropout rate = 0.2: keep rate = 0.8 → nhân với $1 / 0.8 = 1.25$
- Việc nhân bù này giúp độ lớn trung bình của tín hiệu không bị thay đổi quá nhiều.

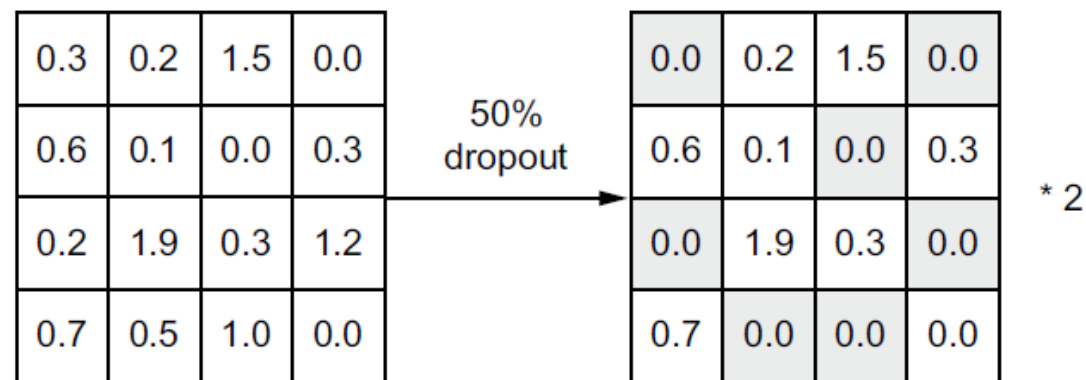


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

5. Tại test time thì sao?

- Khi **test/inference**, dropout **không tắt neuron nữa**.
- Ở test time:
 - Không random đặt activation về 0.
 - Dùng toàn bộ activation.
 - Ma trận activation giữ nguyên.
 - Model dự đoán ổn định, không bị ngẫu nhiên.

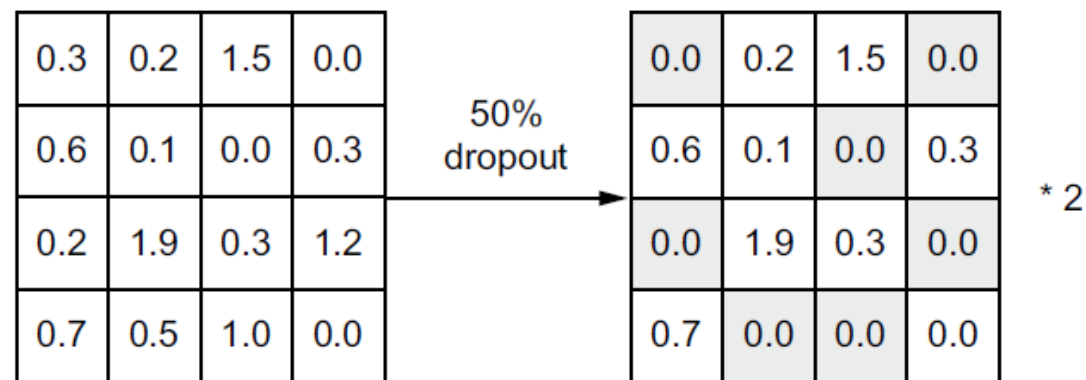


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

6. Vì sao dropout giúp giảm overfitting?

- Dropout buộc model không được phụ thuộc quá nhiều vào một vài neuron cụ thể.
- Nếu không có dropout, model có thể học kiểu:
 - “Chỉ cần neuron A bật là dự đoán class C.”
- Nhưng khi dropout, neuron A có thể bị tắt ngẫu nhiên trong training. Vì vậy model phải học cách phân tán thông tin qua nhiều neuron khác nhau.
- Dropout làm model bớt học thuộc, buộc model học các pattern tổng quát hơn.

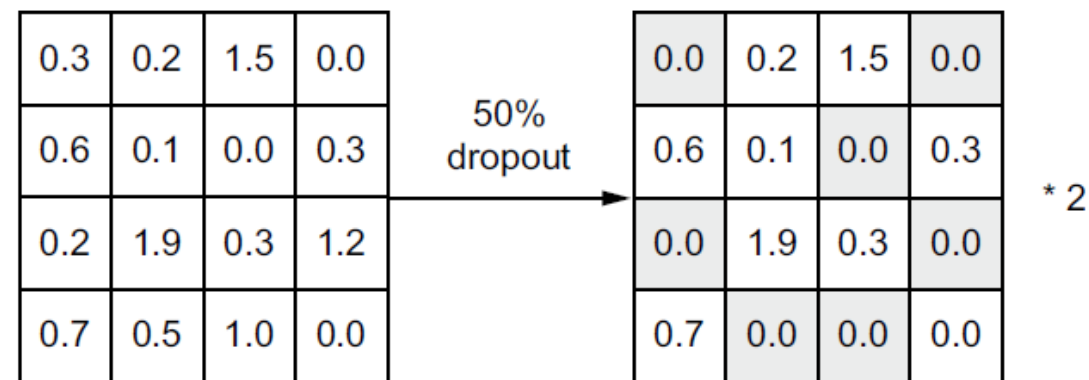


Figure 5.20 Dropout applied to an activation matrix at training time, with rescaling happening during training. At test time the activation matrix is unchanged.

Dropout / Tắt ngẫu nhiên neuron khi huấn luyện

- Model ban đầu học tốt lúc đầu, nhưng sau đó bắt đầu overfit.
- Model có Dropout cũng giảm lúc đầu, sau đó tăng lại, nhưng **tăng chậm hơn nhiều** so với model ban đầu.
- Dropout-regularized model có validation loss ổn định hơn original model.
- Điều này cho thấy Dropout giúp giảm overfitting bằng cách tắt ngẫu nhiên một phần activation trong training.

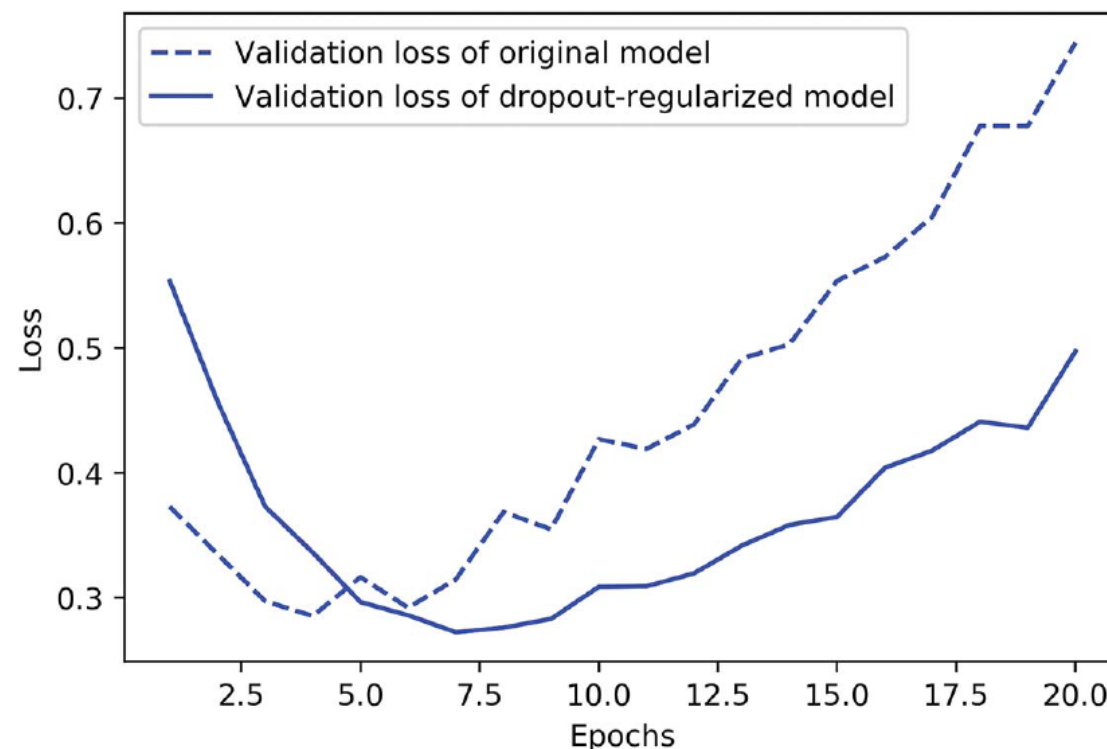


Figure 5.21 Effect of dropout on validation loss

Tổng kết

- Mục tiêu của machine learning là **generalization**
- Cần đánh giá mô hình đúng cách
- Nếu model chưa học đủ → cải thiện **fit**
- Nếu model học quá mức → cải thiện **generalization**
- Dữ liệu, evaluation và regularization quan trọng như nhau

Câu hỏi thảo luận

- Vì sao training accuracy cao không đủ để khẳng định mô hình tốt?
- Khi nào nên nghi ngờ mô hình đang underfit?
- Vì sao phải có validation set riêng?
- Early stopping và regularization có điểm chung gì?
- Trong thực tế, dữ liệu hay mô hình quan trọng hơn?

